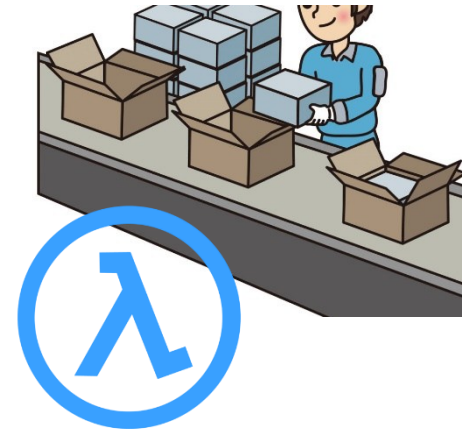




Høgskulen
på Vestlandet



DAT108

Programmering og webapplikasjoner

F04 – Lambda og Streams

Lars-Petter Helland
Bergen
28. august 2026

"Pensum" i dag, videoer + lesestoff

Flere videoer fra javabrainns, ca. 25 minutter:

Navn	Lenge
 Java 8 Lambda Basics 22 - The foreach iteration.mp4	00:09:24
 Java 8 Lambda Basics 23 - Introduction To Streams.mp4	00:11:25
 Java 8 Lambda Basics 24 - More About Streams.mp4	00:06:16

>>> [Lenke til video nr 22](#)

Enda en video med en del gode eksempler, ca. 18 minutter:

<https://www.youtube.com/watch?v=t1-YZ6bF-g0>

Litt lesestoff:

<https://stackify.com/streams-guide-java-8/>

<https://www.baeldung.com/java-8-collectors>

Plan for dagen

Jeg har lagt opp til at vi kan gjøre en eksempler “sammen” i dag!

Utgangspunkt for alle eksemplene er i prosjektet **f04-metoderefs-streams-fyllinn**.

Raskt gjøre ferdig det siste eksempelet fra forrige gang f03:

- › People (`no.hv1.dat108.f03.people`)

Eksempler for å øve på det vi har lært:

- › Eksempel 1-3 (`no.hv1.dat108.f04`)

Streams !!



Eksempler med streams:

- › Eksempel 4-8 (`no.hv1.dat108.f04`)

Fullføre people-eksempelet fra sist

f04-metoderefs-streams-itimen

--- no.hv1.dat108.f03.people



Repetisjon/øvelse - Eksempel1

f04-metoderefs-streams-itimen

--- no.hvl.dat108.f04

--- Eksempel1



Metodereferanser - Eksempel2

f04-metoderefs-streams-itimen

--- no.hvl.dat108.f04

--- Eksempel2



Comparator.comparing

```
sort(personliste, (a,b) -> a.getLastName().compareTo(b.getLastName()));
```

eller

```
sort(personliste, Comparator.comparing(Person::getLastName));
```



Har noen sett denne?
Hva er det for noe?

vedAaSammenligne() - Eksempel3

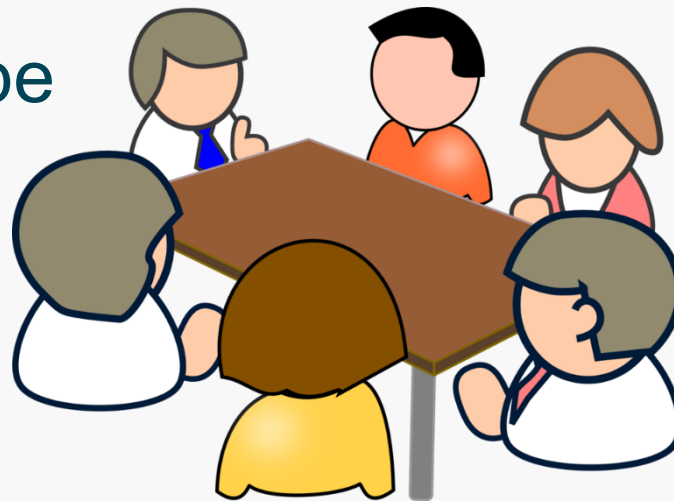
f04-metoderefs-streams-itimen

```
--- no.hvl.dat108.f04
```

Vi ser på bruken av `Comparator.comparing`, og om vi kunne laget en slik selv (om den ikke fantes).



Vi trenger en
referansegruppe





And...
Let's continue!



Imperativ vs. funksjonell programmering

- › **Imperativ stil:** En sekvens av setninger som via endringer i programmets tilstand (variabler) arbeider seg frem til et svar.

```
List<Integer> positiveTall = new ArrayList<>();  
for (int x : enListeMedTall) {  
    if (x > 0) {  
        positiveTall.add(x);  
    }  
}
```

- › **Funksjonell stil:** Nøstede funksjonskall som gir et svar direkte.

```
List<Integer> positiveTall = enListeMedTall .. filter(x -> x > 0) ..;
```

forEach

La oss si at vi ønsker å skrive ut alle elementene i en liste. Hvilke alternativer har vi?

- › Løkker / Eksterne iteratorer. Imperative. Løkkegjennomgang styrt av utvikler.

```
for (int i=0; i<people.size(); i++) {  
    System.out.println(people.get(i));  
}
```

```
for (Person p : people) {  
    System.out.println(p);  
}
```

- › **NY!** Interne iteratorer. Funksjonelle.
Løkkegjennomgang automatisk.

```
people.forEach(p -> System.out.println(p));
```



Streams

- › Nå skal vi se på det som kalles Java sitt Stream-API.
- › Dette gir oss kraftige verktøy for å jobbe med lister eller sekvenser av data
- › Vi skal se at programmer der dere tidligere har brukt 10 kodelinjer nå kan skrives med én enkel kodelinje.
- › Ved å kombinere noen standard metodekall og bruke lambda-uttrykk kan vi skrive programmer på et høyere abstraksjonsnivå, nesten som i SQL. F.eks.:

```
List<String> etternavnene = people.stream()           //FROM people
    .filter(p -> p.getAge() > 50)                   //WHERE age > 50
    .map(Person::getLastName)                        //SELECT lastName
    .toList();
```

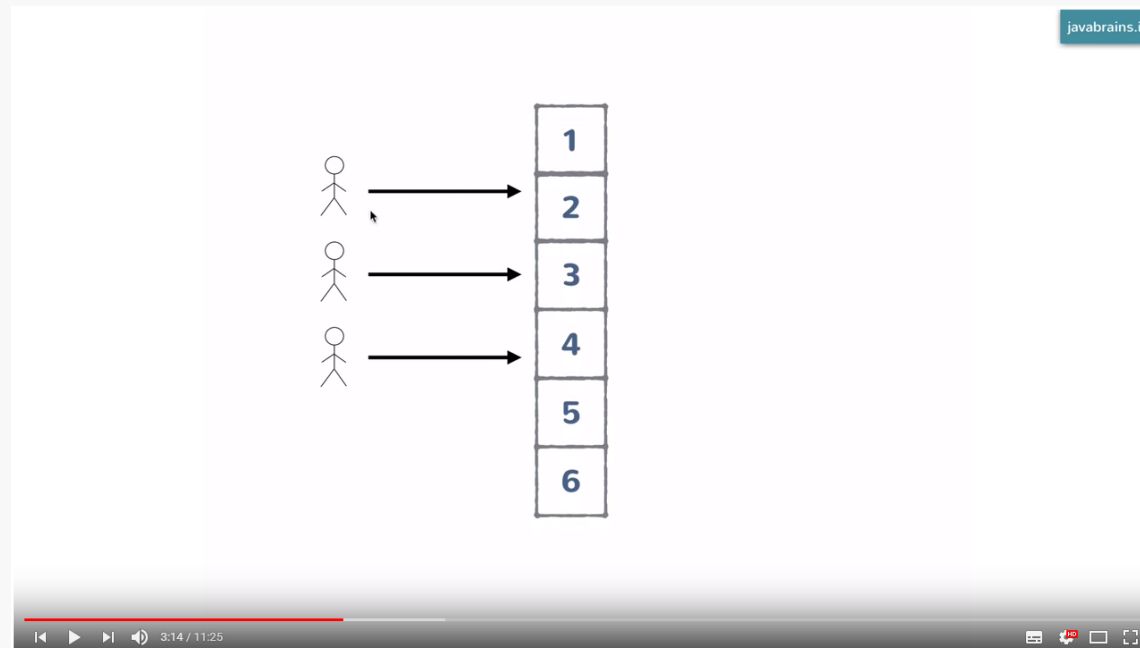
Stream | Pipeline | Conveyor Belt | Assembly Line



Streams - videointroduksjon

Jeg anbefaler at dere ser en del av video 23 **Introduction To Streams** (0:40 - 4:06)

<https://youtu.be/0bHCxjkk0s?list=PLqq-6Pq4ITTa9YGfyhyW2CqdtW9RtY-I3&t=40>



Streams

Vi kan betrakte behandling av samlinger/sekvenser av data med en **samlebånd-analogi**:

- › Vi har en kilde som forsyner samlebåndet med data

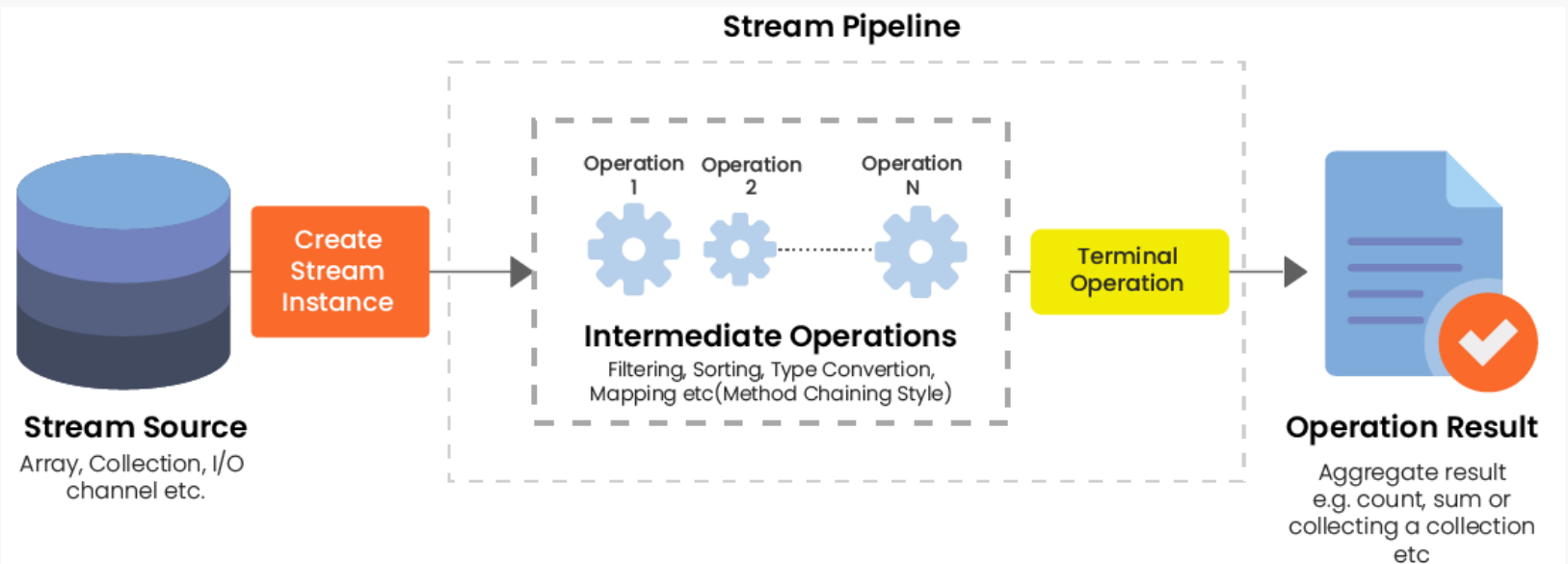
source

- › Vi har én eller flere operasjoner som utføres på elementene underveis på samlebåndet

intermediate operations

- › Vi har én endelig operasjon på enden av samlebåndet som gjør noe med elementene

terminal operation



Eksempel4

Vi tar utgangspunkt i eksemplet vi har jobbet med tidligere, altså at vi har en Person-klasse, og en main() med en liste av personer.

```
public class Eksempel1 {  
  
    public static void main(String[] args) {  
  
        List<Person> people = Arrays.asList(  
            new Person("Charles", "Dickens", 60),  
            new Person("Lewis", "Carroll", 42),  
            new Person("Thomas", "Carlyle", 51),  
            new Person("Charlotte", "Bronte", 45),  
            new Person("Matthew", "Arnold", 39));  
    }  
}
```

La oss nå se hvordan vi kan skrive ut alle med fornavn som begynner på "C".



Kommentarer til løsning

```
people.stream()  
    .filter(p -> p.getLastName().startsWith("C"))  
    .forEach(System.out::println);
```

- › List har en **stream()**-metode som lager en stream med listen som kilde. (I andre språk gjøres dette automatisk, dvs. at en liste også er en stream)
- › Vi får et **Stream<Person>**-objekt
- › Stream har en rekke innebyggede underveis-operasjoner, f.eks. **filter(λ)**
- › Stream har en rekke innebyggede avslutning-operasjoner, f.eks. **forEach(λ)**



Eksempel5

Vi tar en kopi av forrige eksempel og starter derfra.

Nå: **Lage en ny liste av etternavnene til de personene som er over 50 år.**

Vi skal sammenligne litt med SQL. SQL er et deklarativt språk, og regnes som mer "høynivå" enn f.eks. Java. Løsningen i SQL er omtrent slik:

```
SELECT lastName
FROM people
WHERE age > 50;
```

Hvordan løser vi denne oppgaven i Java ("gammel" og "ny" måte)?



Kommentarer til løsning

```
List<String> etternavnene = people.stream()           //FROM people
    .filter(p -> p.getAge() > 50)                   //WHERE age > 50
    .map(Person::getLastName)                        //SELECT lastName
    .toList();
```

- › Vi ser at vi kan skrive "deklarativt", dvs. setter opp **hva** vi ønsker å gjøre i stedet for **hvordan** det skal gjøres.
- › Vi ser at det ligner mye på SQL (selv om rekkefølgen er litt ulik)
- › **map(Function)** er en metode som avbilder en input til en output. I eksemplet avbildes en Person til en String som inneholder etternavnet.
- › **toList()** er en metode (terminal operation) som gjør streamen om igjen til en List. toList() er et spesialtilfelle av den mer genelle collect(Collector).



Nå har vi altså sett et par eksempler på at:

- › Vi har en kilde som forsyner samlebåndet med data

source (`people.stream()..`)

- › Vi har én eller flere operasjoner som utføres på elementene underveis på samlebåndet

intermediate operations (`..filter().., ..map()..`)

- › Vi har én endelig operasjon på enden av samlebåndet som gjør noe med elementene

terminal operation (`..forEach(), ..toList()`)



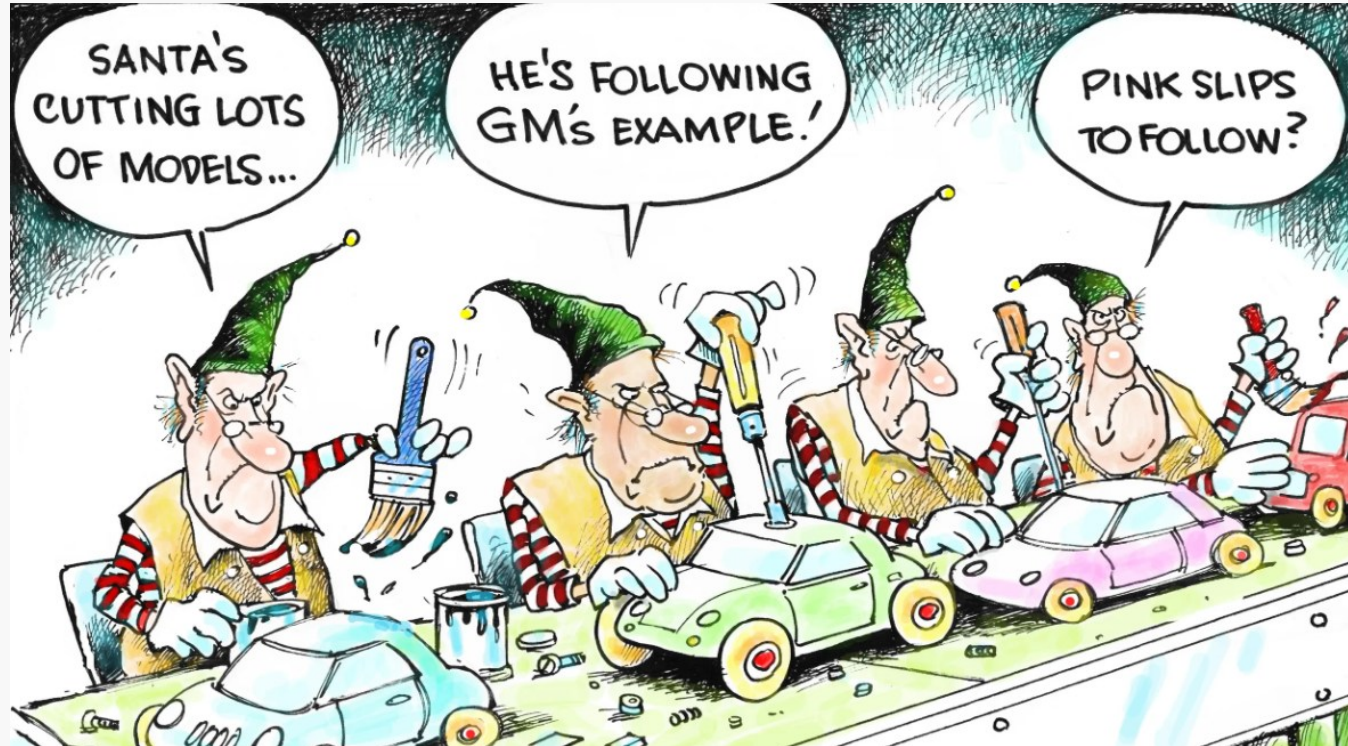
Eksempel6

Vi ser på noen flere mulige kilder til streams:

- › `Stream.of`(tabell eller verdier)
- › `Arrays.stream`(tabell)
- › `liste.stream`()
- › `streng.chars`()
- › `IntStream.range`(min, maks)
- › `IntStream.rangeClosed`(min, maks)
- › `Paths.get`(filsti)
- › ...



Intermediate operations Disse er viktige !!



filter(Predicate<T> p) En intermediate operasjon

- › Vi spanderer et eget lysbilde på filter.
- › Filter er en av de sentrale operasjonene i funksjonell programmering.
- › Filter plukker ut og returnerer en stream med de elementene som oppfyller en betingelse / et predikat.
- › Filter brukes altså til å redusere antall elementer i streamen
- › Filter endrer ikke på selve elementene

```
IntStream.range(1,10)  
    .filter(i -> i % 2 == 0)  
    .forEach(System.out::println);
```



map(Function<T, R> m) En intermediate operasjon

- › Vi spanderer også et eget lysbilde på map.
- › Map er en av de sentrale operasjonene i funksjonell programmering.
- › Map transformerer hvert enkelt element i henhold til en funksjon og returnerer en stream med de ny elementene.
- › Map brukes altså til å transformere elementer i streamen
- › Map endrer ikke antall elementer i streamen

```
people.stream()  
    .map(p -> p.getLastName() + " ")  
    .forEach(System.out::print);
```



Strøm av primitiver

- › **mapToInt() -> IntStream**
- › Denne har metoder som den generelle `Stream<T>` ikke har, f.eks:
 - › `sum()`
 - › `summaryStatistics()`
 - › `average()`
 - › ...
- › `boxed()` -> tilbake til `Stream<Integer>`
- › diverse static source-metoder, f.eks. `of()` og `rangeClosed()`,

(+neste gang; `min()`, `max()`, `findAny()`, `findFirst()`, ...)



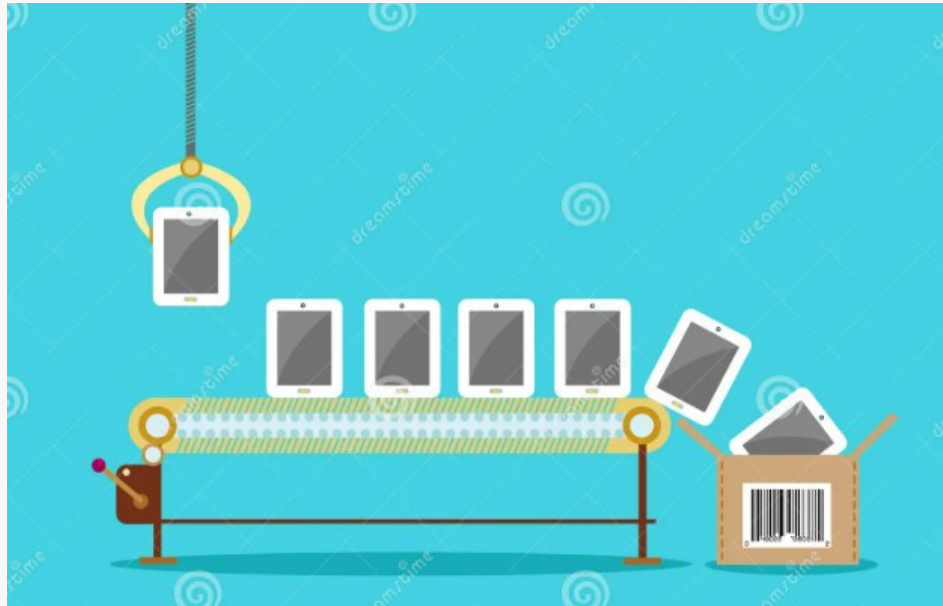
Eksempel7

Vi ser på en del vanlige intermediate operations gjennom noen småeksempler:

- › filter
- › map
- › mapToInt
- › limit
- › distinct
- › sorted



Terminal operations Et par til av disse !!



collect(Collector c)

- › Vi spanderer et eget lysbilde på **collect()**.
- › collect er litt Java-spesifikt. Den brukes f.eks. til å gjøre om streamen til en vanlig collection igjen. collect returnerer da en collection som tilsvarer elementene i streamen. I noen andre språk gjøres dette automatisk. Eksempel (tilsv. `toList()` direkte):

```
List<String> listeAvEtternavn = people.stream()  
    .map(p -> p.getLastName())  
    .collect(Collectors.toList());
```

- › collect kan også brukes til andre ting. Generelt kan vi si at den akkumulerer opp elementene i streamen. Det er mange muligheter med collect, men vi vil ikke gå noe nærmere inn på denne.



reduce(T init, BinaryOperator<T> accu)

- › Vi spanderer et eget lysbilde på reduce.
- › Enda en **terminal operation**. Vi har allerede sett på forEach, toList og collect.
- › På samme måte som **filter** og **map**, er **reduce** en svært sentral metode i funksjonell programmering.
- › Reduce returnerer én verdi basert på elementene i streamen ved å akkumulere opp etter en gitt formel.

```
int sumAlder = people.stream()  
    .map(p -> p.getAge())  
    .reduce(0, (sum, alder) -> sum + alder);
```

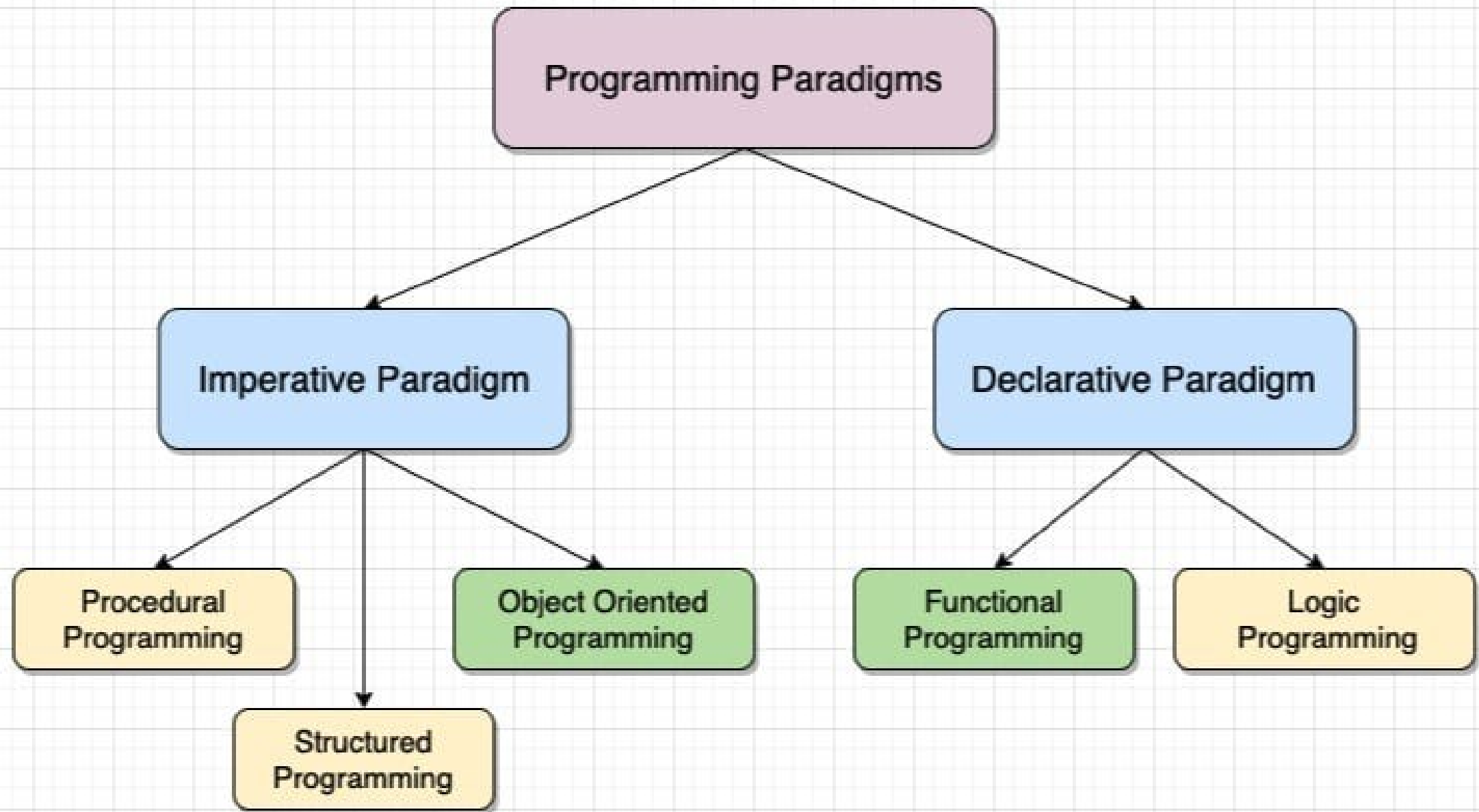
- › Første parameter er startverdi i akkumuleringen
- › Andre parameter er en funksjon som forteller hvordan det skal akkumuleres. Den akkumulerer "i en løkke" til streamen er tom.



Eksempel8

reduce ++





Trenger vi egentlig funksjonell programmering?

Noen fordeler med funksjonell vs. imperativ programmering er at vi får:

- › Kode som er mer **generell / fleksibel** (vi kan parametrisere oppførsel)
- › Kode som er mer **lesbar** (høyere abstraksjonsnivå, færre kodelinjer)
- › Kode som er lettere å **vedlikeholde** (færre kodelinjer, bedre modularitet)
- › Kode som er lettere å **teste** (færre / ingen variabler og sideeffekter)
- › Kode som er lettere å **parallellisere** (kommer tilbake til det)

Videre

Mandag avrunder vi denne modulen med å summere opp, ta et par ting til + snakke litt generelt om **funksjonell programmering**.

Fredag neste uke starter vi på (neste modul i faget) **programmering med tråder**

Oblig1 er lagt ut. Frist søndag 6. september, dvs. om en uke.

